



Name:	Lyka Sophia S. Jardeleza	Date:	02/05/2026
Program:	BSCpE	Section:	A461

A. Guided Exercises:

(Please include the following: 1. Photos of the codes and 2. Outputs after executing the Python code.)

1. Line Model

Photos of Python Codes in Colaboratory	Photo/s of Python Outputs
a) closed-form solution and line-data plot <pre>#Least Squares Method: Closed Form Solution #OPTIMIZATION VIA CLOSED-FORM SOLUTION dataX = np.array(_x) dataY = np.array(_y) # y = ax + b save_wghts = [0.0, 0.0] #Equation (1) #SXY = a*SX2 + b*SXi SXY = np.sum(dataX * dataY) SX2 = np.sum(np.power(dataX, 2)) SXi = np.sum(dataX) #Equation (2) #SYi = a*SXi + b*m SYi = np.sum(dataY) m = len(dataX) #SOLVING FOR coefficient a: save_wghts[0] = (m * SXY - SXi * SYi) / (m * SX2 - SXi ** 2) #SOLVING FOR intercept b: save_wghts[1] = (SYi - save_wghts[0] * SXi) / m print("FINAL WEIGHTS:", save_wghts) #Plot the Linear Model on top of the Datapoints _yp = [] for i in range(len(dataX)): yp = linear_function(dataX[i], save_wghts, basis=0) _yp.append(yp) #Plot the data plt.figure(num=None, figsize=(7, 7)) plt.xlim(int(np.min(dataX)) - 5, int(np.max(dataX)) + 5) plt.ylim(int(np.min(dataY)) - 5, int(np.max(dataY)) + 5) plt.scatter(dataX, dataY) plt.plot(dataX, _yp, color='red') plt.show()</pre>	a) FINAL WEIGHTS: [np.float64(-1.3958988924372626), np.float64(12.467663924802208)]



b) general solution by matrix method

```
#Least Squares Method: General Solution (Matrix Method)

#OPTIMIZATION VIA MATRIX METHOD
dataX = np.array([_x, np.ones(shape=len(_x))])
dataY = np.array([_y])

#Moore-Penrose Pseudoinverse
X2 = np.matmul(dataX, np.transpose(dataX))
p_inv = np.linalg.inv(X2)

XY = np.matmul(dataX, np.transpose(dataY))

w = np.matmul(p_inv, XY)

print("FINAL WEIGHTS: \n", w)
```

b)

```
FINAL WEIGHTS:
[[-1.39589089]
 [12.46766392]]
```

c) SGD solution and line-data plot

```
#Sequential Learning: Stochastic Gradient Descent

#OPTIMIZATION VIA STOCHASTIC GRADIENT DESCENT
data_x = _x
data_y = _y

my_weights = [0.0, 0.0]
save_wgths = [0.0, 0.0]
epoch = 100 #The number of times you go through all the samples
alpha = 0.002 #learning rate (adjust the learning rate if the loss gets very large or blows up to infinity)

min_sqe = 0.0
for i in range(len(data_x)):
    get_diff = data_y[i] - linear_function(data_x[i], my_weights, basis=0)
    min_sqe += get_diff ** 2
min_sqe = 0.5 * min_sqe
print("Initial SQE = ", min_sqe)

for e in range(epoch):
    for i in range(len(data_x)):
        grad = -np.array([data_x[i], 1]) * (data_y[i] - linear_function(data_x[i], my_weights, basis=0))
        my_weights = my_weights - alpha * grad
    #compute loss
    sqe = 0.0
    for j in range(len(data_x)):
        get_diff = data_y[j] - linear_function(data_x[j], my_weights, basis=0)
        sqe += get_diff ** 2
    sqe = 0.5 * sqe

    if min_sqe > sqe:
        min_sqe = sqe
        save_wgths = my_weights

    print("Epoch:", e + 1, " Sample:", i + 1, " SQE =", sqe)

print("\n\n" + "FINAL WEIGHTS:", save_wgths, " MIN SQE:", min_sqe)
```

#Plot the SGD Linear Model on top of the Datapoints

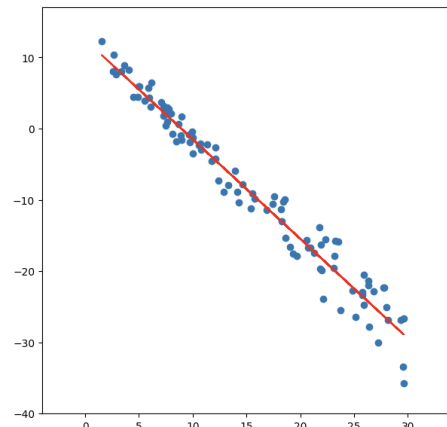
```
_yp = []
for i in range(len(data_x)):
    yp = linear_function(data_x[i], save_wgths, basis=0)
    _yp.append(yp)

#Plot the data
plt.figure(num=None, figsize=(7, 7))
plt.xlim(int(np.min(dataX)) - 5, int(np.max(dataX)) + 5)
plt.ylim(int(np.min(dataY)) - 5, int(np.max(dataY)) + 5)
plt.scatter(data_x, data_y)
plt.plot(data_x, _yp, color='red')
plt.show()
```

c)

```
Initial SQE = 18682.618199999998
Epoch: 1 Sample: 100 SQE = 3882.722662179666
Epoch: 2 Sample: 100 SQE = 3520.145644786684
Epoch: 3 Sample: 100 SQE = 3261.335205183341
Epoch: 4 Sample: 100 SQE = 3024.303887724102
Epoch: 5 Sample: 100 SQE = 2807.229926868023
Epoch: 6 Sample: 100 SQE = 2608.443482774918
Epoch: 7 Sample: 100 SQE = 2426.413990855194
Epoch: 8 Sample: 100 SQE = 2259.738544849731
Epoch: 9 Sample: 100 SQE = 2107.1313609333956
Epoch: 10 Sample: 100 SQE = 1967.4138344420487
Epoch: 11 Sample: 100 SQE = 1839.5057987162427
Epoch: 12 Sample: 100 SQE = 1722.4172337222897
Epoch: 13 Sample: 100 SQE = 1615.2407742634516
Epoch: 14 Sample: 100 SQE = 1517.144823658639
Epoch: 15 Sample: 100 SQE = 1427.3672404999186
Epoch: 16 Sample: 100 SQE = 1345.2095500777068
Epoch: 17 Sample: 100 SQE = 1270.03164278603
Epoch: 18 Sample: 100 SQE = 1201.2469008141481
Epoch: 19 Sample: 100 SQE = 1138.317747715366
Epoch: 20 Sample: 100 SQE = 1080.751555723307
Epoch: 21 Sample: 100 SQE = 1028.0968988233212
Epoch: 22 Sample: 100 SQE = 979.940116031708
Epoch: 23 Sample: 100 SQE = 935.9821663420266
Epoch: 24 Sample: 100 SQE = 895.637319758074
Epoch: 25 Sample: 100 SQE = 858.822580717176
Epoch: 26 Sample: 100 SQE = 825.1711347209335
Epoch: 27 Sample: 100 SQE = 794.4142470146243
Epoch: 28 Sample: 100 SQE = 766.3071623535607
Epoch: 29 Sample: 100 SQE = 740.6266470433048
Epoch: 30 Sample: 100 SQE = 717.1642798924977
Epoch: 31 Sample: 100 SQE = 695.7348724357818
Epoch: 32 Sample: 100 SQE = 676.1650380441498
Epoch: 33 Sample: 100 SQE = 658.2968675287556
Epoch: 34 Sample: 100 SQE = 641.9857201502872
Epoch: 35 Sample: 100 SQE = 627.0991145769535
Epoch: 36 Sample: 100 SQE = 613.5157122957858
Epoch: 37 Sample: 100 SQE = 601.1243857804698
Epoch: 38 Sample: 100 SQE = 589.8233643502035
Epoch: 39 Sample: 100 SQE = 579.519451313132
Epoch: 40 Sample: 100 SQE = 570.1273062742193
Epoch: 41 Sample: 100 SQE = 561.5687874848502
Epoch: 42 Sample: 100 SQE = 553.7723489339207
Epoch: 43 Sample: 100 SQE = 546.6724877989955
Epoch: 44 Sample: 100 SQE = 540.2893288044081
Epoch: 45 Sample: 100 SQE = 534.3277088538647
Epoch: 46 Sample: 100 SQE = 528.9776456158049
Epoch: 47 Sample: 100 SQE = 524.1130676328293
Epoch: 48 Sample: 100 SQE = 519.6918829952623
Epoch: 49 Sample: 100 SQE = 515.6755750652156
Epoch: 50 Sample: 100 SQE = 512.0288995634449
Epoch: 51 Sample: 100 SQE = 508.7196095834606
Epoch: 52 Sample: 100 SQE = 505.7182032321047
Epoch: 53 Sample: 100 SQE = 502.9076935861008
Epoch: 54 Sample: 100 SQE = 500.533396521294
Epoch: 55 Sample: 100 SQE = 498.3027335921191
Epoch: 56 Sample: 100 SQE = 496.2850624962137
Epoch: 57 Sample: 100 SQE = 494.4615056413404
Epoch: 58 Sample: 100 SQE = 492.8140852906749
Epoch: 59 Sample: 100 SQE = 491.32918599836344
Epoch: 60 Sample: 100 SQE = 489.90429323058824
Epoch: 61 Sample: 100 SQE = 488.7847591449113
Epoch: 62 Sample: 100 SQE = 487.7007364815033
Epoch: 63 Sample: 100 SQE = 486.7271631928405
Epoch: 64 Sample: 100 SQE = 485.85399375742634
Epoch: 65 Sample: 100 SQE = 485.0720544484354
Epoch: 66 Sample: 100 SQE = 484.3722603084579
Epoch: 67 Sample: 100 SQE = 483.748092970667
Epoch: 68 Sample: 100 SQE = 483.19344525926317
Epoch: 69 Sample: 100 SQE = 482.69966095815374
Epoch: 70 Sample: 100 SQE = 482.2619318149704
Epoch: 71 Sample: 100 SQE = 481.8749581879881
Epoch: 72 Sample: 100 SQE = 481.533912679265
Epoch: 73 Sample: 100 SQE = 481.2443021247653
Epoch: 74 Sample: 100 SQE = 480.9723849504841
Epoch: 75 Sample: 100 SQE = 480.7447268749219
Epoch: 76 Sample: 100 SQE = 480.5466201098436
Epoch: 77 Sample: 100 SQE = 480.3765053368779
Epoch: 78 Sample: 100 SQE = 480.2311592733705
Epoch: 79 Sample: 100 SQE = 480.10800439479794
Epoch: 80 Sample: 100 SQE = 480.00403803465105
Epoch: 81 Sample: 100 SQE = 479.9195431920802
Epoch: 82 Sample: 100 SQE = 479.8502592044944
Epoch: 83 Sample: 100 SQE = 479.79526988806894
Epoch: 84 Sample: 100 SQE = 479.75302437952683
Epoch: 85 Sample: 100 SQE = 479.7221161820776
Epoch: 86 Sample: 100 SQE = 479.70127043933974
Epoch: 87 Sample: 100 SQE = 479.68932339955015
Epoch: 88 Sample: 100 SQE = 479.6852568247535
Epoch: 89 Sample: 100 SQE = 479.68809831250826
Epoch: 90 Sample: 100 SQE = 479.6970024465687
Epoch: 91 Sample: 100 SQE = 479.7111977065836
Epoch: 92 Sample: 100 SQE = 479.7299880727277
Epoch: 93 Sample: 100 SQE = 479.7527462665401
Epoch: 94 Sample: 100 SQE = 479.7789075741727
Epoch: 95 Sample: 100 SQE = 479.80796420275533
Epoch: 96 Sample: 100 SQE = 479.83946012473064
Epoch: 97 Sample: 100 SQE = 479.8729863687897
Epoch: 98 Sample: 100 SQE = 479.90817671951646
Epoch: 99 Sample: 100 SQE = 479.9447037910335
Epoch: 100 Sample: 100 SQE = 479.98227544285646
```

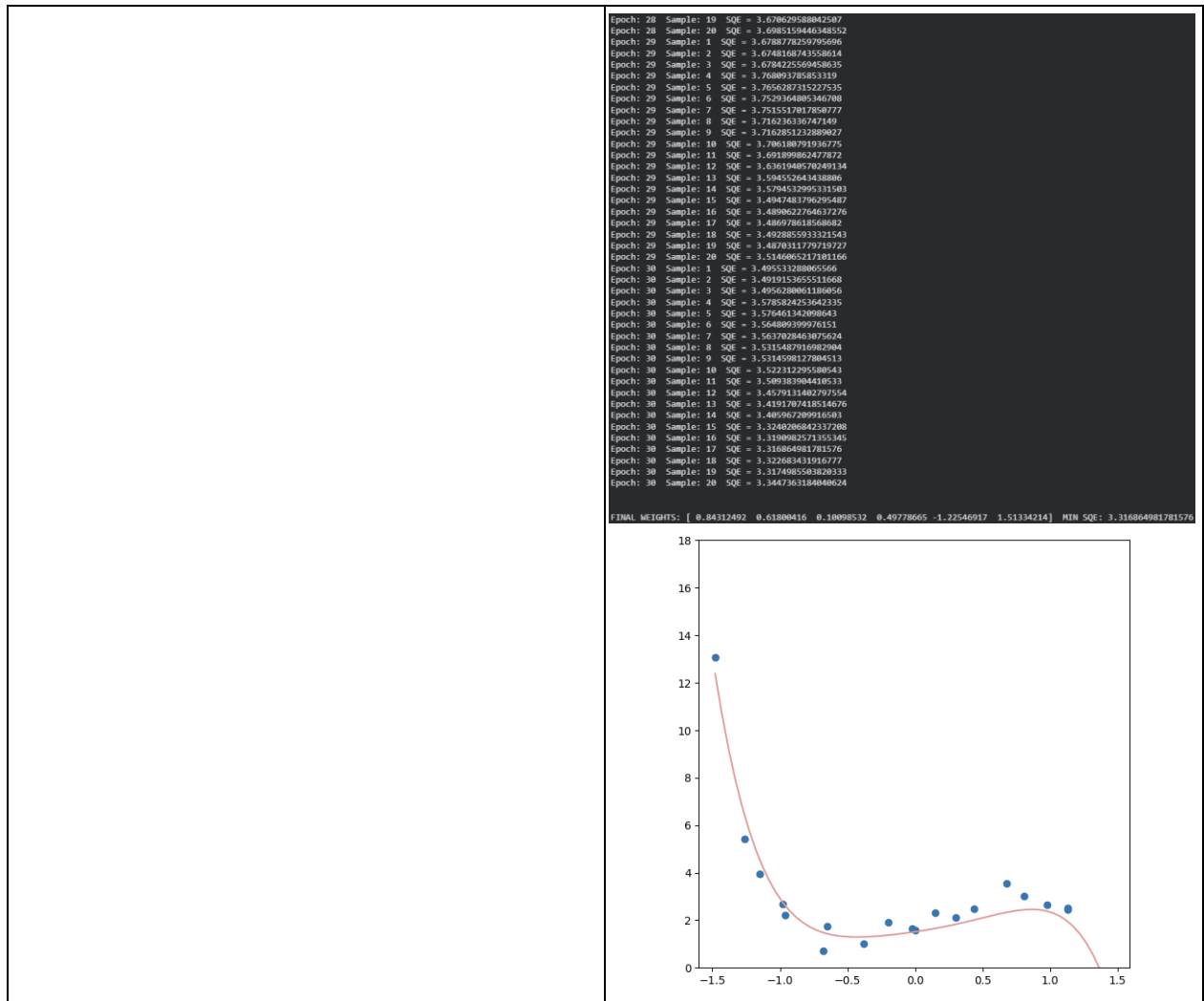
```
FINAL WEIGHTS: [-1.39694801 12.47882064] MIN SQE: 478.10895471714648
```



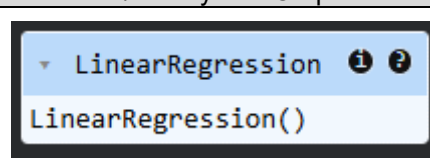


2. Polynomial Model

Photos of Python Codes in Colaboratory	Photo/s of Python Outputs
a) general solution by matrix method <pre>#Polynomial Regression: General Solution (Matrix Method) #OPTIMIZATION VIA MATRIX METHOD x_data = [] get_x = np.array(_x) for i in range(len(get_x)): x_data.append(np.power(get_x, i + 1)) x_data.append(np.ones(shape=len(_x))) dataX = np.array(x_data) dataY = np.array(_y) #Moore_Penrose Pseudoinverse X2 = np.matmul(dataX, np.transpose(dataX)) p_inv = np.linalg.inv(X2) XY = np.matmul(dataX, np.transpose(dataY)) w = np.matmul(p_inv, XY) print("FINAL WEIGHTS:\n", np.transpose(w)[0])</pre>	a) <pre>FINAL WEIGHTS: [1.35330097 0.2370209 0.58995154 0.57965028 -1.63815486 1.8507292]</pre>
b) SGD solution and line-data plot <pre>#Sequential Learning: SGD for Polynomial Model #OPTIMIZATION VIA STOCHASTIC GRADIENT DESCENT data_x = _x data_y = _y coeff_len = 6 my_weights = np.zeros(shape=coeff_len, dtype=np.float64) save_wgths = np.copy(my_weights) epoch = 30 #The number of times you go through all the samples alpha = 0.005 #learning rate (adjust the learning rate if the loss gets very large or blows up to infinity) min_sqe = 0.0 for i in range(len(data_x)): get_diff = data_y[i] - linear_function(data_x[i], my_weights, basis=1) min_sqe += get_diff ** 2 min_sqe = 0.5 * min_sqe print("Initial SQE =", min_sqe) def poly_derivative(x_input, coeff_len): x_deriv = np.zeros(shape=coeff_len, dtype=np.float64) x_deriv[-1] = 1.0 for i in range(coeff_len - 1): x_deriv[i] = np.power(x_input, i + 1) return x_deriv for e in range(epoch): for i in range(len(data_x)): grad = -poly_derivative(data_x[i], coeff_len) * (data_y[i] - linear_function(data_x[i], my_weights, basis=1)) my_weights = my_weights - alpha * grad #Compute loss sqe = 0.0 for j in range(len(data_x)): get_diff = data_y[j] - linear_function(data_x[j], my_weights, basis=1) sqe += get_diff ** 2 sqe = 0.5 * sqe if min_sqe > sqe: min_sqe = sqe save_wgths = my_weights print("Epoch:", e + 1, " Sample:", i + 1, " SQE =", sqe) print("\n\n" + "FINAL WEIGHTS:", save_wgths, " MIN SQE:", min_sqe) #Plot the Polynomial SGD Model on top of the Datapoints plt_x = np.arange(np.min(data_x), np.max(data_x), 0.05) _yp = [] for i in range(len(plt_x)): yp = linear_function(plt_x[i], save_wgths, basis=1) _yp.append(yp) #Plot the data plt.figure(num=None, figsize=(7, 7)) plt.xlim(-1.6, np.max(_x) + 0.1) plt.ylim(0, int(np.max(data_y)) + 5) plt.scatter(data_x, data_y) plt.plot(plt_x, _yp, color='#EB9694') plt.show()</pre>	b) <pre>Initial SQE = 153.27524999999997 Epoch: 1 Sample: 1 SQE = 149.27162474829908 Epoch: 1 Sample: 2 SQE = 148.68711577021668 Epoch: 1 Sample: 3 SQE = 147.49857462737077 Epoch: 1 Sample: 4 SQE = 140.84047292001014 Epoch: 1 Sample: 5 SQE = 140.38708408626366 Epoch: 1 Sample: 6 SQE = 139.7844967712278 Epoch: 1 Sample: 7 SQE = 139.35951595715017 Epoch: 1 Sample: 8 SQE = 138.64428412139748 Epoch: 1 Sample: 9 SQE = 138.2832925404382 Epoch: 1 Sample: 10 SQE = 133.814440209202 Epoch: 1 Sample: 11 SQE = 133.18847041165017 Epoch: 1 Sample: 12 SQE = 133.82642378797146 Epoch: 1 Sample: 13 SQE = 133.04078504865737 Epoch: 1 Sample: 14 SQE = 133.06800159294104 Epoch: 1 Sample: 15 SQE = 113.00959180262872 Epoch: 1 Sample: 16 SQE = 113.72542004496286 Epoch: 1 Sample: 17 SQE = 113.43704269890989 Epoch: 1 Sample: 18 SQE = 104.93704266602876 Epoch: 1 Sample: 19 SQE = 104.52272999213544 Epoch: 1 Sample: 20 SQE = 50.64380901135897 Epoch: 2 Sample: 1 SQE = 50.15452893116448 Epoch: 2 Sample: 2 SQE = 49.848853397278675 Epoch: 2 Sample: 3 SQE = 49.48871541771082 Epoch: 2 Sample: 4 SQE = 49.550785948754594 Epoch: 2 Sample: 5 SQE = 49.30024917905793 Epoch: 2 Sample: 6 SQE = 48.92347700189561 Epoch: 2 Sample: 7 SQE = 48.688648866189666 Epoch: 2 Sample: 8 SQE = 47.8689469172289 Epoch: 2 Sample: 9 SQE = 47.74166093015258 Epoch: 2 Sample: 10 SQE = 47.036540449372986 Epoch: 2 Sample: 11 SQE = 46.587215426738126 Epoch: 2 Sample: 12 SQE = 45.84948050712941 Epoch: 2 Sample: 13 SQE = 45.133997600147254 Epoch: 2 Sample: 14 SQE = 44.61147780292711 Epoch: 2 Sample: 15 SQE = 42.26517653211686 Epoch: 2 Sample: 16 SQE = 41.96280491751965 Epoch: 2 Sample: 17 SQE = 41.99209530369623 Epoch: 2 Sample: 18 SQE = 40.885290425096585 Epoch: 2 Sample: 19 SQE = 40.6098807412023 Epoch: 2 Sample: 20 SQE = 29.60284354989365 Epoch: 3 Sample: 1 SQE = 29.60665344077896 Epoch: 3 Sample: 2 SQE = 29.424365014185643 Epoch: 3 Sample: 3 SQE = 29.289605271795873 Epoch: 3 Sample: 4 SQE = 29.624968897914293 Epoch: 3 Sample: 5 SQE = 29.467219186377697 Epoch: 3 Sample: 6 SQE = 29.20800619467034 Epoch: 3 Sample: 7 SQE = 29.060513573151567 Epoch: 3 Sample: 8 SQE = 28.381752832741874 Epoch: 3 Sample: 9 SQE = 28.328483351656253 Epoch: 3 Sample: 10 SQE = 28.249733863824524 Epoch: 3 Sample: 11 SQE = 27.920993267389917 Epoch: 3 Sample: 12 SQE = 27.05750552282916 Epoch: 3 Sample: 13 SQE = 26.490852789204606 Epoch: 3 Sample: 14 SQE = 25.98448998567972 Epoch: 3 Sample: 15 SQE = 26.081922214415034 Epoch: 3 Sample: 16 SQE = 25.686621813126447</pre>



3. Scikit-Learn Linear Model on Diabetes dataset

Photos of Python Codes in Colaboratory	Photo/s of Python Outputs
<pre> #Linear Regression Using Scikit-Learn With Diabetes Dataset #Import the dataset from sklearn.datasets import load_diabetes diabetes_data = load_diabetes() #Generate the train and test data from sklearn.model_selection import train_test_split X_train, X_test, y_train, y_test = train_test_split(diabetes_data.data, diabetes_data.target, test_size=0.20, random_state=1) #Initialize and fit the Scikit-Learn Linear Model from sklearn.linear_model import LinearRegression skl_linreg = LinearRegression(fit_intercept = True, copy_X = True, n_jobs = None) get_X = X_train get_Y = y_train #Train the model skl_linreg.fit(get_X, get_Y) #Show the Coefficients and Test the Linear Model #The coefficients and intercept of the linear model print("Coeff:", skl_linreg.coef_) print("Intercept:", skl_linreg.intercept_) #Test the Linear Model total_error = 0.0 for i in range(len(X_test)): xtest = X_test[i] ytest = y_test[i] yptest = skl_linreg.predict(np.array([xtest])) sq_error = (ytest - yptest[0]) ** 2 print(i, ": True Target:", ytest, " Estimated Target:", round(yptest[0], 1), " Squared Error:", round(sq_error, 2)) total_error += sq_error average_error = round(total_error / len(X_test), 2) print("Total Error:", round(total_error, 2)) print("Average Error:", average_error) </pre>	



```

Coeff: [ -30.62219046 -272.2506057 528.85252681 327.69716891 -581.01973841
332.97061365 -27.97314298 139.28036115 665.07667773 61.90517166 ]
Intercept: 151.4308932966009
0 : True Target: 78.0 Estimated Target: 119.1 Squared Error: 1692.69
1 : True Target: 152.0 Estimated Target: 110.8 Squared Error: 1699.76
2 : True Target: 200.0 Estimated Target: 185.0 Squared Error: 223.88
3 : True Target: 59.0 Estimated Target: 68.0 Squared Error: 81.18
4 : True Target: 311.0 Estimated Target: 171.1 Squared Error: 19584.76
5 : True Target: 178.0 Estimated Target: 190.4 Squared Error: 154.24
6 : True Target: 332.0 Estimated Target: 218.6 Squared Error: 12861.81
7 : True Target: 132.0 Estimated Target: 119.6 Squared Error: 152.92
8 : True Target: 156.0 Estimated Target: 157.3 Squared Error: 1.61
9 : True Target: 135.0 Estimated Target: 129.5 Squared Error: 30.24
10 : True Target: 220.0 Estimated Target: 214.2 Squared Error: 34.21
11 : True Target: 233.0 Estimated Target: 202.9 Squared Error: 904.64
12 : True Target: 91.0 Estimated Target: 88.4 Squared Error: 6.66
13 : True Target: 51.0 Estimated Target: 72.0 Squared Error: 439.54
14 : True Target: 195.0 Estimated Target: 236.9 Squared Error: 1754.48
15 : True Target: 109.0 Estimated Target: 206.1 Squared Error: 9427.76
16 : True Target: 217.0 Estimated Target: 175.6 Squared Error: 1713.52
17 : True Target: 94.0 Estimated Target: 102.0 Squared Error: 63.44
18 : True Target: 89.0 Estimated Target: 114.1 Squared Error: 630.49
19 : True Target: 111.0 Estimated Target: 169.0 Squared Error: 3369.29
20 : True Target: 129.0 Estimated Target: 187.7 Squared Error: 3447.41
21 : True Target: 181.0 Estimated Target: 85.6 Squared Error: 9092.64
22 : True Target: 168.0 Estimated Target: 144.1 Squared Error: 569.51
23 : True Target: 97.0 Estimated Target: 107.1 Squared Error: 101.39
24 : True Target: 115.0 Estimated Target: 94.0 Squared Error: 438.97
25 : True Target: 202.0 Estimated Target: 211.8 Squared Error: 96.01
26 : True Target: 84.0 Estimated Target: 90.7 Squared Error: 45.27
27 : True Target: 147.0 Estimated Target: 176.2 Squared Error: 854.44
28 : True Target: 253.0 Estimated Target: 122.6 Squared Error: 16998.19
29 : True Target: 144.0 Estimated Target: 180.9 Squared Error: 1363.45
30 : True Target: 262.0 Estimated Target: 157.1 Squared Error: 11000.66
31 : True Target: 115.0 Estimated Target: 138.0 Squared Error: 530.99
32 : True Target: 68.0 Estimated Target: 192.4 Squared Error: 15466.21
33 : True Target: 65.0 Estimated Target: 94.5 Squared Error: 867.58
34 : True Target: 252.0 Estimated Target: 166.3 Squared Error: 7343.02
35 : True Target: 212.0 Estimated Target: 189.4 Squared Error: 509.11
36 : True Target: 142.0 Estimated Target: 110.7 Squared Error: 981.14
37 : True Target: 215.0 Estimated Target: 247.7 Squared Error: 1067.85
38 : True Target: 180.0 Estimated Target: 172.5 Squared Error: 56.65
39 : True Target: 163.0 Estimated Target: 188.1 Squared Error: 629.49
40 : True Target: 151.0 Estimated Target: 160.1 Squared Error: 82.53
41 : True Target: 283.0 Estimated Target: 176.7 Squared Error: 11309.65
42 : True Target: 66.0 Estimated Target: 115.8 Squared Error: 2481.23
46 : True Target: 302.0 Estimated Target: 152.4 Squared Error: 22370.69
47 : True Target: 93.0 Estimated Target: 144.1 Squared Error: 2611.01
48 : True Target: 178.0 Estimated Target: 190.2 Squared Error: 148.6
49 : True Target: 241.0 Estimated Target: 191.5 Squared Error: 2450.74
50 : True Target: 52.0 Estimated Target: 59.8 Squared Error: 61.03
51 : True Target: 144.0 Estimated Target: 159.8 Squared Error: 248.28
52 : True Target: 102.0 Estimated Target: 102.1 Squared Error: 0.02
53 : True Target: 200.0 Estimated Target: 147.9 Squared Error: 2715.41
54 : True Target: 232.0 Estimated Target: 186.1 Squared Error: 2106.44
55 : True Target: 97.0 Estimated Target: 121.4 Squared Error: 595.95
56 : True Target: 109.0 Estimated Target: 159.1 Squared Error: 2512.32
57 : True Target: 55.0 Estimated Target: 83.4 Squared Error: 808.74
58 : True Target: 63.0 Estimated Target: 55.6 Squared Error: 55.08
59 : True Target: 98.0 Estimated Target: 79.1 Squared Error: 357.85
60 : True Target: 88.0 Estimated Target: 99.5 Squared Error: 131.27
61 : True Target: 233.0 Estimated Target: 199.9 Squared Error: 1095.93
62 : True Target: 235.0 Estimated Target: 168.5 Squared Error: 4416.08
63 : True Target: 97.0 Estimated Target: 109.2 Squared Error: 147.94
64 : True Target: 243.0 Estimated Target: 276.4 Squared Error: 1116.59
65 : True Target: 59.0 Estimated Target: 76.5 Squared Error: 306.62
66 : True Target: 138.0 Estimated Target: 74.1 Squared Error: 4078.47
67 : True Target: 220.0 Estimated Target: 168.0 Squared Error: 2705.03
68 : True Target: 137.0 Estimated Target: 198.2 Squared Error: 3749.23
69 : True Target: 72.0 Estimated Target: 58.8 Squared Error: 174.83
70 : True Target: 109.0 Estimated Target: 163.5 Squared Error: 2975.64
71 : True Target: 71.0 Estimated Target: 87.4 Squared Error: 270.15
72 : True Target: 74.0 Estimated Target: 89.6 Squared Error: 242.41
73 : True Target: 219.0 Estimated Target: 135.8 Squared Error: 6923.15
74 : True Target: 196.0 Estimated Target: 158.8 Squared Error: 1385.65
75 : True Target: 170.0 Estimated Target: 94.3 Squared Error: 5734.64
76 : True Target: 199.0 Estimated Target: 108.1 Squared Error: 8271.7
77 : True Target: 71.0 Estimated Target: 85.6 Squared Error: 212.1
78 : True Target: 155.0 Estimated Target: 223.4 Squared Error: 4672.8
79 : True Target: 52.0 Estimated Target: 176.5 Squared Error: 15510.81
80 : True Target: 63.0 Estimated Target: 100.0 Squared Error: 1369.23
81 : True Target: 88.0 Estimated Target: 110.8 Squared Error: 519.21
82 : True Target: 97.0 Estimated Target: 149.7 Squared Error: 2776.62
83 : True Target: 100.0 Estimated Target: 152.4 Squared Error: 2743.83
84 : True Target: 64.0 Estimated Target: 113.3 Squared Error: 2435.39
85 : True Target: 107.0 Estimated Target: 111.4 Squared Error: 19.78
86 : True Target: 49.0 Estimated Target: 98.4 Squared Error: 2435.88
87 : True Target: 60.0 Estimated Target: 73.1 Squared Error: 171.72
88 : True Target: 346.0 Estimated Target: 270.5 Squared Error: 5699.57
Total Error: 266339.73
Average Error: 2992.58

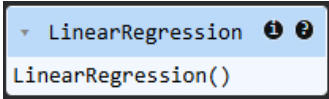
```



B. Problem Exercises:

(Please include the following: 1. Photos of the codes and 2. Outputs after executing the Python code.)

Problem 1:

Photo/s of Python output	Photos of Python Codes in Colaboratory
a) 	a) <pre>#CODING PROBLEM 1 import numpy as np from sklearn.datasets import load_diabetes from sklearn.model_selection import train_test_split from sklearn.linear_model import LinearRegression # Load dataset diabetes = load_diabetes() X = diabetes.data y = diabetes.target # Generate train and test data X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=7) # a.) skl_model = LinearRegression(fit_intercept=True) skl_model.fit(X_train, y_train)</pre>
b) <pre>(np.float64(263978.4060953839), np.float64(2966.049506689707))</pre>	b) <pre># b.) # Predict using test data y_pred = skl_model.predict(X_test) # Compute squared errors squared_errors = (y_test - y_pred) ** 2 total_squared_error = np.sum(squared_errors) average_squared_error = np.mean(squared_errors) total_squared_error, average_squared_error</pre>
c) <pre>Matrix intercept: 153.96248367788561 Sklearn intercept: 153.96248367788561 Matrix coefficients: [-55.61758436 -259.84103752 540.30814766 344.62440892 -971.66791394 612.39201814 155.14489732 190.63749388 830.74389027 73.87020866] Sklearn coefficients: [-55.61758436 -259.84103752 540.30814766 344.62440892 -971.66791394 612.39201814 155.14489732 190.63749388 830.74389027 73.87020866]</pre>	c) <pre># c.) # Center the data (this is what Scikit-Learn does internally) X_mean = np.mean(X_train, axis=0) y_mean = np.mean(y_train) Xc = X_train - X_mean yc = y_train - y_mean # Solve for coefficients (no bias term) coefficients_matrix = np.linalg.pinv(Xc) @ yc # Recover intercept intercept_matrix = y_mean - np.dot(X_mean, coefficients_matrix) intercept_matrix, coefficients_matrix print("Matrix intercept:", intercept_matrix) print("Sklearn intercept:", skl_model.intercept_) print("\nMatrix coefficients:\n", coefficients_matrix) print("\nSklearn coefficients:\n", skl_model.coef_)</pre>



d) <pre># d.) # Initialize parameters alpha = 0.0001 # learning rate epochs = 100 # Add bias term X_train_bias = np.c_[np.ones(X_train.shape[0]), X_train] X_test_bias = np.c_[np.ones(X_test.shape[0]), X_test] # Initialize weights w_sgd = np.zeros(X_train_bias.shape[1]) # SGD training for epoch in range(epochs): for i in range(X_train_bias.shape[0]): xi = X_train_bias[i] yi = y_train[i] prediction = np.dot(xi, w_sgd) error = prediction - yi w_sgd = w_sgd - alpha * error * xi</pre>	d)
e) <pre># e.) # Predict using SGD model y_pred_sgd = X_test_bias @ w_sgd # Compute squared errors squared_errors_sgd = (y_test - y_pred_sgd) ** 2 total_squared_error_sgd = np.sum(squared_errors_sgd) average_squared_error_sgd = np.mean(squared_errors_sgd) total_squared_error_sgd, average_squared_error_sgd</pre>	e) <pre>(np.float64(435094.20436690556), np.float64(4888.698925470849))</pre>

C. Findings ,Observations, and Comments:

Guiding Questions :

1. What did you learn in the guided exercises? Did you learn something new?
2. What are your findings, observations, and comments in the guided exercises?
3. What are your findings, observations, and comments in the problem exercises?
4. Did you experience any difficulties in solving the problem exercises?

Yes, I learned how linear regression works step by step. I understood how the model uses data to make predictions and how coefficients and intercept affect the results. I also learned that linear regression can be solved in different ways, not just using built-in functions.

The guided exercises were helpful because they explained the process clearly. They made it easier to understand the flow before doing the coding problems. I noticed that breaking the steps down helped me avoid confusion.

In the problem exercises, I observed that using different methods (Scikit-Learn and matrix method) can give the same results if done correctly. This helped me understand that the math behind the model is the same even if the approach is different.

Yes, I had some difficulty at first, especially in matching my matrix results with the Scikit-Learn output. I also got confused with variable names and outputs, but after checking and comparing the results, I was able to fix it.